

Beyond Gradient Descent: Yakubovich's Stripe Algorithms for Finite-Update Learning[★]

$a, *, 1, \quad b, 2$

$a, , , , ,$
 $b, , , , ,$

ARTICLE INFO

Keywords:

linear inequalities
Stripe algorithm
Kernel extensions
Projection methods
Finite convergence
Supervised learning

ABSTRACT

Projection-based algorithms based on recurrent systems of linear inequalities, commonly known as Stripe algorithms, originate in the study of V. A. Yakubovich on finitely convergent learning procedures for pattern recognition and binary classification. Although Yakubovich already considered such methods in the early development of machine learning, including their connection to perceptron-type learning through his eponymous algorithm YAVA, they have received relatively little attention in the modern machine learning literature, especially in the context of kernel methods and nonlinear decision functions.

In this paper, we show that such methods can be naturally formulated within standard supervised learning settings. We consider two complementary learning formulations: an L -system, related to squared-loss optimization via relaxed optimality conditions, and a C -system, defined by pointwise tolerance constraints on predictions.

Unlike traditional gradient-based optimizers, Stripe algorithms are governed by iterative projection updates that trigger only upon constraint violation. Under strict feasibility assumptions, the framework admits theoretical guarantees for finite-update convergence, yielding stable and predictable training dynamics that eliminate the need for learning-rate scheduling or heuristic early stopping.

We also introduce a kernelized extension of the framework, which enables nonlinear decision functions through kernel representations by operating in the space of kernel expansion coefficients while preserving the original projection-based update scheme.

Experiments on a broad collection of real-world tabular OpenML datasets demonstrate that Stripe algorithms achieve performance comparable to, and in many cases outperform, strong standard linear and kernel baselines, including SGD and kernel SVM, while exhibiting structured and sparse update dynamics. These results position Stripe algorithms as a simple, stable, and resource-efficient alternative to conventional optimization-based learning methods.

1. Introduction

In supervised learning, model parameters are traditionally determined by minimizing the empirical risk using gradient-based methods.

Despite their broad applicability, such approaches typically require careful tuning of algorithmic hyperparameters, including the learning rate, learning-rate schedules, regularization strength, and stopping criteria. Moreover, in online (streaming) settings, gradient-based methods may be sensitive to noise and to the order in which training examples are presented.

In contrast, the Stripe framework considered in this work is based on Yakubovich's method of recurrent objective inequalities and relies on tolerance and relaxation parameters that define admissible hyperslab regions in the parameter space; these parameters regulate feasibility of the constraint system rather than the optimization trajectory and therefore differ in nature from learning-rate tuning, scheduling, and early stopping used in gradient-based methods.

These practical drawbacks motivate interest in alternative learning paradigms in which parameter updates are driven by explicit feasibility requirements rather than by repeated minimization of a smooth loss function. In such formulations, training is guided by constraint satisfaction conditions, and parameter corrections are applied when these

★

*Corresponding author

 (); ()

 (); ()

ORCID(s):

1

conditions are violated, rather than being defined purely by gradient steps Detassis, Lombardi and Milano (2021); Peters (2023).

Constraint-based formulations interpret learning as the problem of satisfying a system of constraints that defines an admissible region in parameter or output space. In such settings, the goal is not necessarily the minimization of a smooth global loss function, but rather the attainment of feasibility with respect to a collection of equalities or inequalities.

In Yakubovich's formulation Yakubovich (1966), these constraints take the form of hyperslabs in parameter space, and learning proceeds through successive projections driven by constraint violations. Related ideas appear in more recent machine learning frameworks. For example, CQNet formulates training as a split feasibility problem, seeking a parameter vector x such that $x \in C$ and $Ax \in Q$, thereby interpreting learning as motion toward the intersection of convex sets in parameter and output spaces Peters (2023).

Similarly, the Moving Targets framework Detassis et al. (2021) introduces a master–student scheme in which a dedicated correction step enforces prediction constraints before the model parameters are updated to fit the corrected targets. In this setting, constraint satisfaction may take precedence over pure loss minimization.

In this study, we investigate projection-based learning algorithms defined through recurrent objective inequalities, known as Stripe algorithms, originally proposed by V. A. Yakubovich Yakubovich (1965, 1966). Historically, these algorithms were closely connected with early learning problems in pattern recognition and binary classification. Yakubovich's early work considered finitely convergent procedures for constructing separating hyperplanes and for adjusting perceptron-type decision rules. The YAVA algorithm is described geometrically as a projection of the current weight vector onto a hyperplane associated with a newly presented image, and related algorithms are discussed as generalizations of the Rosenblatt perceptron Yakubovich (1966); Fradkov and Shepeljavyi (2022). This places Stripe algorithms in the same historical line of feasibility-based classification procedures, perceptron-type learning, and recurrent objective inequalities. These methods were further developed in control theory and adaptive systems Fomin (1976); Fomin, Fradkov and Yakubovich (1981). In modern machine learning literature, such methods appear relatively rarely. However, there are works where the Stripe algorithm is applied to linear model training and online learning and is compared with standard gradient-based methods. These studies report competitive accuracy and fast convergence compared to linear models trained with stochastic gradient descent Lipkovich (2022); Lipkovich and Sagatdinov (2023).

V.A. Yakubovich introduced several ways to reduce learning problems to systems of parameter inequalities, including constraints derived from relaxed stationarity conditions of a quadratic loss function and constraints defined through pointwise bounds on the prediction error Yakubovich (1966). Following Yakubovich's terminology, we refer to these two types of constraint systems as the L -system and the C -system, respectively. Both formulations lead to systems of linear inequalities and employ a common projection-based parameter update mechanism, differing only in the source of the constraints.

Unlike gradient-based methods, Stripe algorithms update the parameters only when the corresponding constraint is violated, and each update is performed as a projection correction. In the linear case, under strict feasibility of the constraint system, a finite number of updates can be guaranteed Yakubovich (1966).

In practice, this often leads to stable and predictable learning dynamics and lower sensitivity to step-size selection compared to gradient methods. Stripe algorithms still use their own tolerance parameters and stopping rules, but the update mechanism itself is not based on learning-rate scheduling. The behavior of the algorithms in noisy settings, where strict feasibility is violated, is discussed in a later section.

In addition, we consider a kernelized extension of the framework. In this setting, the Stripe constraints and projection-based updates are transferred to kernel representations, enabling nonlinear decision functions in the original input space while preserving the same update scheme.

In this setting, learning is performed in the space of kernel expansion coefficients, while preserving the projection-based form of the updates and enabling fully iterative training, in contrast to standard SVM solvers that typically rely on batch optimization of a fixed objective.

We empirically evaluate the proposed approach on a broad collection of real-world tabular datasets from OpenML¹. Our task selection partially overlaps with the benchmark suites of Bischl, Casalicchio, Feurer, Gijsbers, Hutter, Lang, Mantovani, van Rijn and Vanschoren (2021) and the OpenML AutoML Benchmark Gijsbers, LeDell, Thomas, Poirier, Bischl and Vanschoren (2019). We compare Stripe algorithms with strong standard baseline methods, including

¹<https://www.openml.org>

SGD and SVMs, and show that the projection-based approach achieves competitive performance and in many cases outperforms these baselines while maintaining a simple and robust implementation.

The paper is organized as follows. In Section 2, we formally introduce the supervised learning problem and describe two basic formulations—the L - and C -systems—which form the foundation of the projection-based approach. In Section 3, we present a unified Stripe algorithmic framework, including the canonical form of the constraints, a geometric interpretation, and variants of projection-based updates. Theoretical properties, including conditions for a finite number of updates, are discussed in the corresponding section and in the appendix. In Section 5, **we consider a kernelized extension of the proposed approach, which enables nonlinear decision functions through kernel representations**. An experimental evaluation on real-world tabular OpenML datasets is presented in Section 6, where Stripe algorithms are compared with standard baseline methods.

The implementation of the proposed Stripe algorithms is publicly available at: <https://github.com/nadyashanarova/yakubovich-stripe-learning>.

2. Problem Setting

2.1. Supervised learning setup

We consider a standard supervised learning setting. Let $D = \{(x_i, y_i)\}_{i=1}^m$, or, in the online setting, an unbounded stream $\{(x_i, y_i)\}_{i=1}^\infty$ denote the training data, where $x_i \in \mathcal{X} \subseteq \mathbb{R}^n$, $y_i \in \mathcal{Y}$, with $\mathcal{X}$ denoting the input space and $\mathcal{Y}$ the output (label) space. Here, x_i is an input feature vector and y_i the corresponding target value.

We consider a parametric model $f : \mathcal{X} \times \Theta \rightarrow \mathbb{R}$ with trainable parameter vector $\theta \in \Theta \subseteq \mathbb{R}^p$. In general, the supervised learning setting encompasses both regression and classification problems. In regression, $\mathcal{Y} \subseteq \mathbb{R}$, and the model output $f(x, \theta)$ is interpreted directly as a numerical prediction. In classification, $\mathcal{Y}$ is a finite set of class labels, and the model defines a real-valued scoring function whose value is mapped to a class decision.

In this work, however, the proposed projection-based Stripe framework and all subsequent theoretical developments are considered *exclusively* for binary classification problems. Accordingly, we assume that $y_i \in \{-1, +1\}$. Regression problems are mentioned only as part of the general supervised learning context and are not addressed algorithmically in this paper.

Model parameters are estimated iteratively using a projection-based learning procedure. We denote by θ_k the parameter vector at iteration k and assume that the algorithm is initialized from an arbitrary initial estimate $\theta_1 \in \mathbb{R}^n$ (for example, $\theta_1 = \mathbf{0}$).

In the binary classification setting, the learned scoring function is converted into a class prediction using a thresholding rule

$$\hat{y}(x) = \text{sign}(f(x, \theta) - \tau), \quad (1)$$

where $\tau \in \mathbb{R}$ is a classification threshold. The threshold τ is selected using a validation set and does not participate in the training dynamics of the model.

In the linear setting, we consider a model that is linear in the input features and takes the form $f(x, \theta) = \langle \theta, x \rangle$, where $\langle \cdot, \cdot \rangle$ denotes the standard inner product in $\mathbb{R}^n$. In this case, the model output is a linear function of the input vector x .

Although extending linear models with manually constructed nonlinear features, such as polynomial expansions, can increase their expressive power, this approach typically requires careful selection of the feature degree and quickly suffers from the curse of dimensionality when features are generated explicitly Bishop (2006); Hastie, Tibshirani and Friedman (2009).

To model nonlinear relationships between the input features and the target variable in a principled and scalable manner, we employ a mapping of the input data into an extended, potentially high-dimensional, feature space, as discussed in Section 5.

2.2. L -systems: learning as relaxed optimality

The first basic formulation corresponds to the minimization of a quadratic loss function. For a finite training dataset D , this leads to the empirical risk $Q(\theta) = \frac{1}{m} \sum_{i=1}^m (y_i - f(x_i, \theta))^2$.

In the standard optimization setting, the parameter vector θ is determined from the stationarity condition $\nabla_\theta Q(\theta) = \mathbf{0}$.

Following the classical formulation introduced by Yakubovich Yakubovich (1966), we adopt a *relaxed* version of the first-order optimality condition: instead of enforcing exact stationarity, small deviations of the gradient components from zero are allowed.

In this formulation, learning is posed as the satisfaction of a system of inequality constraints, commonly referred to as an *L*-system in the classical literature.

In particular, the relaxed optimality conditions can be written explicitly as

$$\left| \frac{\partial Q(\theta)}{\partial \theta_j} \right| \leq \varepsilon, \quad j = 1, \dots, n, \quad (2)$$

where $\varepsilon > 0$ is a prescribed tolerance.

The tolerance parameter ε specifies the admissible deviation from exact first-order optimality and defines the width of the corresponding hyperslab constraints in the parameter space. In Yakubovich (1966), ε reflects the level of irreducible error or noise in the data and controls feasibility rather than optimization accuracy.

In this study, ε is treated as a problem-dependent hyperparameter; its practical selection is discussed in Section 6.

As will be shown in Section 3, for a linear parametrization $f(x, \theta) = \langle \theta, x \rangle$, the relaxed optimality conditions (2) are equivalent to a system of linear inequalities in canonical form. These inequalities define stripe-shaped (hyperslab) constraints in the parameter space, which can be processed using projection-based algorithms.

We refer to this constraint-based formulation as the *L*-**system**, following the terminology of the classical stripe algorithm literature. The *L*-system applies naturally to regression problems with a quadratic loss. When used for classification, the quadratic loss is employed as a surrogate objective for learning a real-valued scoring function; we do not consider a probabilistic or likelihood-based interpretation in this setting. From this viewpoint, the *L*-system provides a projection-based alternative to standard gradient optimization methods.

2.3. C-systems: learning as constraint satisfaction

The second basic formulation considered in this paper is based not on minimizing a functional, but on enforcing *pointwise constraints* on the prediction error. Specifically, for each observation the model is required to satisfy $|y - f(x, \theta)| \leq \varepsilon$, where $\varepsilon > 0$ is a prescribed tolerance.

Following the formulation introduced by Yakubovich, learning posed in this form corresponds to the satisfaction of a system of inequality constraints. Throughout the paper, we refer to this formulation as the *C*-**system**.

In regression problems, it corresponds to uniform approximation within an ε -tube, while in classification problems it enforces control over the deviation of the prediction from the target label in terms of an ε -insensitive region.

Unlike *L*-systems, the existence of a solution to a *C*-system is not guaranteed without additional assumptions and depends on the feasibility of the constraint set. Nevertheless, *C*-systems naturally arise in online learning scenarios and in settings where controlling the maximum error is more important than minimizing its mean squared value.

2.4. Kernelized extension

Although linear models are limited to representing linearly separable relationships in the original feature space, this limitation is traditionally addressed by kernel methods, which enable nonlinear decision functions through an implicit mapping to an extended feature space.

In this work, we employ this classical idea in a different context and propose a kernelized extension of the projection-based Stripe framework. Unlike standard approaches, where kernelization is applied to gradient-based or loss-minimization algorithms, here it is used to transfer the system of stripe-shaped constraints and the associated projection-based updates to the space of kernel expansion coefficients.

This kernelized extension preserves the geometric interpretation and update structure of Stripe algorithms and allows the same projection-based scheme to be applied through kernel representations.

To model nonlinear dependencies between the input features and the target function, we employ the standard kernel construction based on an implicit mapping of the original feature space $\mathbb{R}^n$ into an extended (possibly infinite-dimensional) Hilbert feature space $\phi : \mathcal{X} \subseteq \mathbb{R}^n \rightarrow \mathcal{H}$. **This transformation does not alter the original learning formulation and serves purely as a technical device for representing nonlinear decision functions in a form that is linear in the feature-space parameters.**

In this representation, the model takes the form $f(x, \theta) = \langle \theta, \phi(x) \rangle$, where θ denotes a parameter vector in the feature space $\mathcal{H}$. By the Representer Theorem (Schölkopf and Smola (2002); Hofmann, Schölkopf and Smola (2008)),

solutions of regularized empirical risk minimization problems in a reproducing kernel Hilbert space (RKHS) admit a finite kernel expansion representation. In particular, using the reproducing property of RKHS, the reproducing kernel $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ satisfies $K(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}}$, and the model can be implemented without explicit computation of the feature mapping $\phi(x)$.

The kernelized variant is therefore viewed as a practical extension that preserves the geometric interpretation and the projection-based update structure of Stripe algorithms in the space of kernel expansion coefficients.

As shown below, both learning formulations (the L - and C -systems) lead to the sequential processing of stripe-shaped constraints in the parameter space and can be addressed within a unified projection-based algorithmic framework, referred to as Stripe.

3. Projection-Based Stripe Algorithms

This section describes the projection-based Stripe algorithmic framework used to solve the systems of constraints arising from the L - and C -learning formulations. The exposition focuses on the canonical form of the constraints, the geometric interpretation of the updates, and a unified form of the projection step.

3.1. Canonical form of the constraints

Both the L - and C -formulations lead to the sequential processing of constraints of the canonical form

$$\left| \langle a_k, \theta \rangle + b_k \right| < \varepsilon, \quad k = 1, 2, \dots, \quad (3)$$

where $a_k \in \mathbb{R}^n$ are coefficient vectors, $b_k \in \mathbb{R}$ are scalar offsets, and $\varepsilon > 0$ is a fixed tolerance. In the batch setting, the index set is finite ($k = 1, \dots, m$); the infinite index notation is used to accommodate the online case.

Inequality (3) defines a stripe-shaped admissible region in the parameter space. More precisely, the equality $\langle a_k, \theta \rangle + b_k = 0$ defines the central hyperplane $H_k = \{\theta \in \mathbb{R}^n : \langle a_k, \theta \rangle + b_k = 0\}$. The inequality $|\langle a_k, \theta \rangle + b_k| < \varepsilon$ defines the corresponding stripe, or hyperslab, $S_k = \{\theta \in \mathbb{R}^n : |\langle a_k, \theta \rangle + b_k| < \varepsilon\}$, bounded by the two parallel hyperplanes $\langle a_k, \theta \rangle + b_k = \pm \varepsilon$. Thus, a hyperplane corresponds to an exact equality constraint, whereas a stripe/hyperslab represents an admissible tolerance region around that hyperplane.

We introduce the residual

$$\eta_k(\theta) := \langle a_k, \theta \rangle + b_k, \quad (4)$$

so that (3) is equivalent to $|\eta_k(\theta)| < \varepsilon$.

3.1.1. Relation to the C - and L -formulations

In the C -system, each constraint is generated by a single observation. In the linear case $f(x, \theta) = \langle \theta, x \rangle$, we set $a_k = x_i$, $b_k = -y_i$, which yields the residual $\eta_k(\theta) = \langle a_k, \theta \rangle + b_k = \langle \theta, x_i \rangle - y_i$.

In the L -system, the constraints are derived from relaxed stationarity conditions of the quadratic loss function $Q(\theta) = \frac{1}{m} \sum_{i=1}^m (y_i - \langle \theta, x_i \rangle)^2$.

In the linear case, the gradient can be written in matrix form as $\nabla Q(\theta) = 2(A^{(m)}\theta - b^{(m)})$, where

$$A^{(m)} := \frac{1}{m} \sum_{i=1}^m x_i x_i^\top + \lambda I, \quad b^{(m)} := \frac{1}{m} \sum_{i=1}^m y_i x_i.$$

Applying relaxed component-wise constraints to $\nabla Q(\theta)$ yields a system of linear inequalities of the canonical form (3), where the vectors a_k correspond to rows of the matrix $A^{(m)}$, and the scalars b_k are given by the corresponding components of $b^{(m)}$.

3.2. Unified projection step

Stripe algorithms perform parameter corrections *only when the corresponding constraint is violated*. Learning is viewed as an iterative process of searching for a feasible parameter vector in the space $\mathbb{R}^n$.

At each iteration, the current constraint of the canonical form $|\eta_k(\theta)| = |\langle a_k, \theta \rangle + b_k| < \varepsilon$ is evaluated. If the constraint is satisfied, no update is performed. Otherwise, a projection-based correction is applied.

Formally, at iteration k , let $\eta_k(\theta_k) = \langle a_k, \theta_k \rangle + b_k$ denote the residual associated with the current constraint. If $|\eta_k(\theta_k)| \geq \varepsilon$, the parameters are updated according to

$$\theta_{k+1} = \theta_k - \gamma_k \frac{\Psi(\eta_k(\theta_k), \varepsilon, \delta, \beta)}{\|a_k\|^2} a_k, \quad (5)$$

where $\varepsilon > 0$ is the tolerance parameter defining the hyperslab width, γ_k is a relaxation parameter, and Ψ denotes a generic correction function.

Equation (5) serves as a unified template encompassing several Stripe algorithms. Different variants correspond to different choices of the correction function Ψ , while preserving the same projection-based update structure.

The specific forms of the correction function Ψ considered in this study are introduced in the following sections:

- Basic Stripe algorithm (Section 3.4): $\Psi(\eta) = \eta$;
- Improved Stripe algorithm (Section 3.5): $\Psi(\eta) = \eta - \varepsilon \text{sign}(\eta)$;
- Multizone C-system algorithm (Section 3.6): $\Psi = \Psi_{\text{mz}}$.

Additional parameters such as δ and β appear only in multizone correction schemes and are introduced in Section 3.6.

3.3. Batch and online settings

The Stripe algorithm admits both a *batch* formulation, in which the constraint system is induced by a fixed finite training dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^m$, and an *online* (streaming) formulation, where observations are received sequentially and incrementally augment the set of constraints.

The distinction between these two settings pertains solely to the manner in which constraints are generated and indexed. In the batch setting, the index set is finite and constraints are processed by iterating over this set, potentially using deterministic or randomized orderings. In the online setting, each incoming observation introduces a new constraint, which is incorporated and processed as soon as it becomes available.

Crucially, this distinction does not affect the structure of the projection update in (5). The update rule itself remains identical across settings; only the mechanism by which the constraint index k is produced—either through iteration over a finite index set or through incremental index generation—differs between the batch and online formulations.

3.4. Basic Stripe algorithm

Definition 1 (Basic Stripe algorithm: projection onto the central hyperplane). *The basic variant corresponds to the choice $\Psi(\eta, \varepsilon, \delta, \beta) = \eta$, and $\gamma_k \equiv 1$ in (5). In this case, the parameters δ and β are inactive.*

Equivalently, the update rule takes the form

$$\theta_{k+1} = \begin{cases} \theta_k, & \text{if } |\eta_k(\theta_k)| < \varepsilon, \\ \theta_k - \frac{\eta_k(\theta_k)}{\|a_k\|^2} a_k, & \text{if } |\eta_k(\theta_k)| \geq \varepsilon. \end{cases} \quad (6)$$

The algorithm (6) admits the following geometric interpretation. If the current iterate θ_k lies outside the hyperslab, the update (6) corresponds to the orthogonal projection onto the central hyperplane $\eta_k(\theta) = 0$. **Indeed, substituting the update rule (6) into the residual definition (4) yields $\eta_k(\theta_{k+1}) = 0$, confirming that the update projects onto the central hyperplane $\langle a_k, \theta \rangle + b_k = 0$ (or equivalently $\langle a_k, \theta \rangle = -b_k$).** This interpretation underlies the term “stripe” used in the classical literature (see, e.g., Yakubovich (1966); Fomin et al. (1981)).

The finite-update convergence analysis of the basic Stripe algorithm relies on a strict feasibility assumption: there exists a parameter vector that satisfies all hyperslab constraints with a positive margin. This assumption is inherent to the classical proof of finite convergence, as it guarantees that successive projections eventually enter the interior of the feasible region.

In practical learning problems with noisy data or model mismatch, strict feasibility may be violated, meaning that no parameter vector satisfies all constraints simultaneously with nonzero slack. To address this situation, Yakubovich proposed a modified variant based on an ε -insensitive zone.

3.5. Improved Stripe algorithm

Definition 2 (Improved Stripe algorithm: projection onto the nearest boundary). *To relax the implicit assumption of strict feasibility of the constraint system, which underlies the finite-convergence analysis of the basic Stripe algorithm and may be violated in the presence of noise or modeling errors, Yakubovich proposed a modified variant based on an ε -insensitive zone.*

In this case, $\Psi(\eta, \varepsilon, \delta, \beta) = \eta - \varepsilon \text{sign}(\eta)$, with the relaxation parameter $\gamma_k \equiv \mu_k \in [1/2, 1]$. In this variant, the parameters δ and β are inactive.

The resulting update rule takes the form

$$\theta_{k+1} = \begin{cases} \theta_k, & \text{if } |\eta_k(\theta_k)| \leq \varepsilon, \\ \theta_k - \mu_k \frac{\eta_k(\theta_k) - \varepsilon \text{sign}(\eta_k(\theta_k))}{\|a_k\|^2} a_k, & \text{if } |\eta_k(\theta_k)| > \varepsilon. \end{cases} \quad (7)$$

The algorithm (7) admits geometric interpretation as well. For $\mu_k = 1$, the update corresponds to a projection onto the nearest boundary of the hyperslab, that is, onto the hyperplane $|\langle a_k, \theta \rangle + b_k| = \varepsilon$. Compared to the basic variant, this leads to more conservative updates and improves the robustness of the algorithm in noisy settings. For $\mu_k \in (1/2, 1)$ the update corresponds to a relaxed projection step; the choice of μ_k affects the aggressiveness of the correction but does not alter the qualitative form of the update rule.

For linear systems of stripe constraints, classical results due to Yakubovich show that projection-based stripe updates terminate after a finite number of corrections, provided that the constraint system is strictly feasible. The following theorem was proved in Yakubovich (1966).

Theorem 1 (Finite convergence, Yakubovich). *Assume that the sequence of feature vectors $\{a_k\}$ is bounded, $\|a_k\| \leq \bar{a} < \infty$, and that there exists an “ideal” parameter vector θ_* such that, for some $\rho \in (0, 1)$, the strict feasibility condition $|\langle a_k, \theta_* \rangle + b_k| < \rho\varepsilon$ for all k holds. Then the improved Stripe algorithm (7) with relaxation parameters $\mu_k \in [1/2, 1]$ is finitely convergent. Moreover, the number of correction steps r (i.e., iterations for which $\theta_{k+1} \neq \theta_k$) satisfies*

$$r \leq \frac{\|\theta_1 - \theta_*\|^2 \bar{a}^2}{\varepsilon^2(1 - \rho)^2}. \quad (8)$$

Remark 1. *The convergence conditions differ for the two versions of the Stripe algorithm. For the basic Stripe algorithm (6) (projection onto the centerline), finite convergence requires the stronger margin condition $\rho < 1/2$.*

In this case, the number of corrective updates satisfies $r_{\text{base}} \leq \frac{\|\theta_1 - \theta_\|^2 \bar{a}^2}{\varepsilon^2(1 - 2\rho)}$.*

In contrast, for the improved ε -insensitive variant considered in (7), this restriction is relaxed, and finite convergence is guaranteed for any $\rho \in (0, 1)$. As a result, the improved algorithm is more suitable for learning problems in the presence of significant noise.

The update rule (5) encompasses, as special cases, the classical algorithms for learning C -systems introduced by V. A. Yakubovich. In the C -system formulation, learning is posed as recurrent processing of stripe-shaped objective inequalities, and parameter updates are performed only when the corresponding constraint is violated. In the classical theory, finite convergence of C -systems is established for specific piecewise-defined correction rules, such as the C_ε and C_5 algorithms. In this setting, finite convergence is understood in the strong sense: there exists an iteration index q such that $\theta_q = \theta_{q+1} = \theta_{q+2} = \dots$, that is, the parameter updates terminate after a finite number of corrections. This result holds under the standard assumptions of strict feasibility of the system of object inequalities and boundedness of the input vectors. These classical algorithms can be interpreted as particular instances of (5) corresponding to specific choices of the correction function Ψ . A more detailed description of the C -system formulation and the associated multizone projection rules is provided in Section 3.6.

3.5.1. Relation to the Kaczmarz method

From a computational perspective, the basic Stripe algorithm is closely related to the classical Kaczmarz method. Although the two methods address different problem formulations, their update rules are algebraically connected.

In Kacmarz (1937), S. Kacmarz proposed an iterative method for solving a fixed system of linear equations of the form $\langle a_k, \theta \rangle + b_k = 0$, $k = 1, \dots, m$, by sequential orthogonal projections of the current iterate onto the corresponding hyperplanes. Given the k -th equation, the Kacmarz update takes the form

$$\theta_{k+1} = \theta_k - \frac{\langle a_k, \theta_k \rangle + b_k}{\|a_k\|^2} a_k. \quad (9)$$

This update coincides algebraically with the correction step of the basic Stripe algorithm (6) on iterations where a violation occurs. The essential difference lies not in the update formula itself, but in the underlying problem formulation.

The Kacmarz method aims to find an exact solution in the intersection of hyperplanes, $\theta_* \in \bigcap_k \{\theta \in \mathbb{R}^n \mid \langle a_k, \theta \rangle + b_k = 0\}$ and therefore continues to update the parameters indefinitely when the system is inconsistent or noisy.

In contrast, the Stripe framework replaces equality constraints by *interval constraints*, $|\langle a_k, \theta \rangle + b_k| < \varepsilon$, which define hyperslabs in the parameter space. Once the current iterate enters the corresponding slab, no further correction is applied for that constraint. This mechanism naturally suppresses oscillatory behavior typical of row-action methods applied to inconsistent systems and leads to more stable learning dynamics.

From this viewpoint, the basic Stripe algorithm can be seen as a feasibility-based relaxation of the Kacmarz method, where the goal is to reach the intersection of hyperslabs rather than hyperplanes. This interpretation aligns Stripe algorithms with modern online and adaptive learning settings, in which constraints may be revealed sequentially and exact consistency is neither assumed nor required.

A related perspective was proposed in Gusev and Bondarko (2020) in the context of adaptive control, where the interaction between the learning algorithm and the environment is modeled as a game in which constraints are generated adaptively in response to the current parameter vector.

3.6. C-systems: multizone projections onto hyperslabs

In the C -formulation, learning is posed as the sequential processing of the canonical hyperslab constraints introduced earlier in (3). In the linear case $f(x, \theta) = \langle \theta, x \rangle$, each observation (x_i, y_i) gives rise to a constraint of the form $|\langle x_i, \theta \rangle - y_i| < \varepsilon$.

In terms of the canonical form (3), this corresponds to the choice $a_k = x_i$, $b_k = -y_i$, with the residual $\eta_k(\theta) = \langle a_k, \theta \rangle + b_k = \langle x_i, \theta \rangle - y_i$.

In the C -system, constraints are processed sequentially, one per iteration (e.g., in a fixed order or using a random permutation of the training set). At iteration k , the residual $\eta_k(\theta_k) = \langle a_k, \theta_k \rangle + b_k$ is evaluated, and a parameter correction is performed only if the corresponding hyperslab constraint is violated, i.e., when $|\eta_k(\theta_k)| \geq \varepsilon$.

The C -system is solved using the same unified Stripe projection step (5) as the L -system. The difference lies exclusively in the choice of the correction function Ψ . In the scheme used in our experiments, we fix $\gamma_k \equiv 1$ and introduce a multizone correction function Ψ_{mz} , which depends on the magnitude of the constraint violation.

In the C -system, parameter updates are performed using the unified Stripe projection step (5) with $\gamma_k \equiv 1$ and a multizone correction function $\Psi(\cdot)$, which depends on the magnitude of the constraint violation. The function $\Psi(\cdot)$ is defined piecewise as

$$\Psi(\eta) = \begin{cases} 0, & |\eta| < \varepsilon, \\ \beta(\eta - \varepsilon \operatorname{sign}(\eta)), & \varepsilon \leq |\eta| < \varepsilon + \delta, \\ 2(\eta - \varepsilon \operatorname{sign}(\eta)), & \varepsilon + \delta \leq |\eta| < 2\varepsilon, \\ \eta, & |\eta| \geq 2\varepsilon. \end{cases}$$

Here $\delta \in (0, \varepsilon)$ specifies the width of the buffer zone, and $\beta \in (0, 2]$ is a relaxation parameter.

If $|\eta_k| < \varepsilon$, the current iterate θ_k lies inside the hyperslab and no correction is performed. For $|\eta_k| \geq 2\varepsilon$, the update coincides with the basic Stripe step (6), that is, with the orthogonal projection onto the central hyperplane $\eta_k(\theta) = 0$. In the intermediate zones, the update corresponds to a reflection with respect to the nearest hyperslab boundary or to a relaxed correction, which improves the robustness of the algorithm in noisy settings.

The multizone correction used in our experiments can be viewed as a simplified variant of the C_ε algorithm proposed by V. A. Yakubovich. It corresponds to formula (5.3) in Yakubovich (1966) in a special case where the stabilizing term $\rho_{j(k)}$ is omitted and the relaxation parameter β is fixed.

4. Application to Machine Learning: Linear Models

In this section, we consider the linear model $f(x, \theta) = \langle \theta, x \rangle$, and demonstrate how the L - and C -learning formulations lead to systems of hyperslab constraints of the canonical form (3). To avoid redundancy, we do not repeat the general supervised learning setup or the basic Stripe update scheme, and instead focus on the instantiation of the coefficients (a_k, b_k) and on practical training procedures.

4.1. Learning L -systems: quadratic loss

We consider a quadratic loss function with ℓ_2 regularization, $Q(\theta) = \frac{1}{m} \sum_{i=1}^m (y_i - \langle \theta, x_i \rangle)^2 + \lambda \|\theta\|^2$.

The components of the gradient are given by $\frac{\partial Q(\theta)}{\partial \theta_j} = 2 \left(\sum_{k=1}^n A_{jk}^{(m)} \theta_k - B_j^{(m)} \right)$, where

$$A_{jk}^{(m)} = \frac{1}{m} \sum_{i=1}^m x_{ij} x_{ik} + \lambda \delta_{jk}, \quad (10)$$

$$B_j^{(m)} = \frac{1}{m} \sum_{i=1}^m y_i x_{ij}. \quad (11)$$

The relaxed stationarity conditions (2) are equivalent to a system of hyperslab constraints $|\langle a_j, \theta \rangle + b_j| < \varepsilon, j = 1, \dots, n$, where $a_j := A_{j\cdot}^{(m)}, b_j := -B_j^{(m)}$. Thus, in the L -system, the constraint index is naturally identified with the index of the parameter coordinate.

In the online setting, the statistics $A^{(m)}$ and $B^{(m)}$ are updated recurrently as new observations arrive. After each update of the statistics, the system of hyperslab constraints changes accordingly, which enables the use of Stripe projection algorithms to maintain approximate feasibility within the prescribed tolerance ε .

4.2. Learning C -systems: pointwise constraints

In the C -formulation, learning is reduced to the sequential processing of pointwise hyperslab constraints on the prediction error (see the detailed description and geometric interpretation in Section 3.6). For the linear model $f(x, \theta) = \langle \theta, x \rangle$, each training example (x_i, y_i) induces a constraint of the form $|\langle x_i, \theta \rangle - y_i| < \varepsilon$. That is, in the canonical form (3), one can set $a_i = x_i, b_i = -y_i$, with the residual given by $\eta_i(\theta) = \langle x_i, \theta \rangle - y_i$.

The C -system is solved by sequentially processing the canonical constraints associated with the training examples. The order in which the constraints are processed (e.g., a single pass or multiple passes over a finite dataset) is determined by the chosen training setting and does not affect the form of the projection step.

At iteration k , the residual $\eta_k(\theta_k) = \langle a_k, \theta_k \rangle + b_k$ is evaluated, and a correction is performed only when the hyperslab is violated, that is, when $|\eta_k(\theta_k)| \geq \varepsilon$. The update takes the form $\theta_{k+1} = \theta_k - \frac{\Psi_{\text{mz}}(\eta_k(\theta_k))}{\|a_k\|^2} a_k$, where the correction function Ψ_{mz} (with buffer-zone width δ and relaxation parameter β) is defined in Section 3.6.

5. Kernelized Extension of Stripe Algorithms

This section introduces a kernelized extension of the projection-based Stripe framework. The extension preserves the geometric interpretation of hyperslab constraints and the structure of projection-based updates, while enabling nonlinear decision functions through kernel representations.

5.1. Kernelized parametrization

We consider a model in the form of a kernel expansion, $f(x; \alpha) = \sum_{k=1}^M \alpha_k K(\tilde{x}_k, x)$, $\alpha \in \mathbb{R}^M$, where $K(\cdot, \cdot)$ is a positive definite kernel and $\{\tilde{x}_k\}_{k=1}^M$ denotes a dictionary (a set of support points). In the batch setting, one typically has $M = m$ and $\tilde{x}_k = x_k$, while in the online setting M may grow as new data arrive.

5.2. Kernelized L -system

Kernel parametrization and dictionary. Let a training dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^m$ be given. For the kernelized model, we employ the representation $f(x; \alpha) = \sum_{k=1}^M \alpha_k K(\tilde{x}_k, x)$, where $\{\tilde{x}_k\}_{k=1}^M$ denotes a *dictionary* of support points. In the batch setting, we take $M = m$ and $\tilde{x}_k = x_k$, while in the online setting M corresponds to the current dictionary size (see B.3).

We consider a quadratic loss function with ℓ_2 regularization in the space of expansion coefficients:

$$Q(\alpha) = \frac{1}{m} \sum_{i=1}^m \left(y_i - \sum_{k=1}^M \alpha_k K(\tilde{x}_k, x_i) \right)^2 + \lambda \|\alpha\|^2. \quad (12)$$

For convenience, we introduce the kernel feature matrix $\Phi \in \mathbb{R}^{m \times M}$ with entries $\Phi_{ik} := K(\tilde{x}_k, x_i)$. Then (12) can be rewritten as $Q(\alpha) = \frac{1}{m} \|y - \Phi \alpha\|^2 + \lambda \|\alpha\|^2$.

The stationary condition $\nabla_{\alpha} Q(\alpha) = 0$ is equivalent to the linear system

$$\left(\frac{1}{m} \Phi^T \Phi + \lambda I \right) \alpha = \frac{1}{m} \Phi^T y. \quad (13)$$

The components of the gradient are given by

$$\frac{\partial Q(\alpha)}{\partial \alpha_j} = 2 \left(\sum_{k=1}^M G_{jk}^{(m,M)} \alpha_k - g_j^{(m,M)} \right), \quad (14)$$

where

$$G_{jk}^{(m,M)} := \frac{1}{m} \sum_{i=1}^m K(\tilde{x}_j, x_i) K(\tilde{x}_k, x_i) + \lambda \delta_{jk}, \quad (15)$$

$$g_j^{(m,M)} := \frac{1}{m} \sum_{i=1}^m y_i K(\tilde{x}_j, x_i). \quad (16)$$

Here δ_{jk} denotes the Kronecker delta.

Instead of enforcing (13) exactly, we adopt a relaxed stationarity condition: $\left| \frac{\partial Q(\alpha)}{\partial \alpha_j} \right| \leq \varepsilon, j = 1, \dots, M$. Using (14), this condition is equivalent to a system of hyperslab constraints of the canonical form (3): $|\langle a_j, \alpha \rangle + b_j| < \varepsilon, j = 1, \dots, M$, where $a_j := G_j^{(m,M)}$, $b_j := -g_j^{(m,M)}$. Thus, the kernelized L -system reduces to a system of linear hyperslab constraints in the space of coefficients α , to which the Stripe projection algorithms can be applied directly.

5.3. Kernelized C -system

In the C -formulation, each observation (x_i, y_i) gives rise to a constraint $|y_i - f(x_i; \alpha)| < \varepsilon$. Substituting the kernelized representation $f(x; \alpha) = \sum_{k=1}^M \alpha_k K(\tilde{x}_k, x)$, we obtain $\left| \sum_{k=1}^M \alpha_k K(\tilde{x}_k, x_i) - y_i \right| < \varepsilon$. This constraint has the canonical form (3), $|\langle a_i, \alpha \rangle + b_i| < \varepsilon$, where $(a_i)_k := K(\tilde{x}_k, x_i)$, $b_i := -y_i$.

This formulation is a direct analogue of the linear C -system, with the input feature vector x_i replaced by the vector of kernel evaluations $(K(\tilde{x}_1, x_i), \dots, K(\tilde{x}_M, x_i))$.

Consequently, the kernelized C -system also reduces to a system of linear hyperslab constraints in the space of coefficients α , to which the Stripe projection algorithms can be applied directly.

6. Experimental Evaluation

6.1. Objectives

The experimental study is designed to assess Stripe methods from three complementary perspectives. First, we evaluate predictive performance in comparison with strong linear and kernel-based baselines on a diverse collection of tabular datasets. Second, we analyze the internal training dynamics, with particular emphasis on the number of corrective updates and the sparsity of the learning process. Third, we investigate the robustness of Stripe with respect

to variations in its structural parameters, including the tolerance level ε , the buffer width δ , and the correction scaling coefficient β .

Unlike gradient-based methods, Stripe does not minimize a global loss function through iterative gradient descent. Instead, it seeks feasibility with respect to a system of stripe constraints and performs parameter updates only when admissibility is violated. As a result, the learning trajectory is governed by geometric feasibility conditions rather than by learning-rate tuning. This distinction motivates an empirical examination of convergence behavior, update sparsity, and the trade-off between model complexity and predictive accuracy.

6.2. Datasets and Preprocessing

The main ranking benchmark is conducted on 53 real-world binary classification tasks from OpenML. Datasets with non-binary target variables are excluded. Additional diagnostic experiments use either subsets of this benchmark or extended collections of datasets, as stated explicitly in the corresponding subsections.

We employ stratified 5-fold cross-validation for performance estimation. Numerical features are imputed and normalized using a quantile transformation. Categorical features are encoded via one-hot encoding. All preprocessing steps are fitted exclusively on the training split and subsequently applied to the validation and test splits to prevent information leakage.

All comparisons within each diagnostic experiment are performed under a common preprocessing and evaluation protocol.

6.3. Compared Methods

We evaluate Stripe under two complementary experimental settings: a purely linear benchmark and a kernel-based benchmark.

Linear benchmark. We compare the following linear classifiers: (i) SGDClassifier with logistic loss (denoted as SGD), (ii) Stripe-L (Linear), and (iii) Stripe-C (Linear).

Kernel benchmark. To assess kernel-based representations, we additionally consider: (i) SVM (kernel), (ii) Stripe-L (kernel), (iii) Stripe-C (kernel), and (iv) RFF-SGD (Random Fourier Features followed by SGD).

6.4. Evaluation Protocol

Hyperparameters are selected using Bayesian optimization with the Tree-structured Parzen Estimator (TPE) [Bergstra, Bardenet, Bengio and Kégl \(2011\)](#), implemented via the hyperopt library. For each dataset and model, the optimization budget is fixed to 20 trials.

To reduce computational cost, hyperparameter optimization is performed on a stratified subsample of at most 500 training instances. Within this subsample, model configurations are evaluated using 3-fold stratified cross-validation, and the objective is the validation F_1 score. The best configuration is subsequently retrained on the full training set.

In the main ranking benchmark, epoch-based methods are trained for at most 20 epochs. Additional diagnostic experiments, including update-dynamics and structural ε -sensitivity analyses, use the epoch budgets stated in the corresponding subsections. Batch-style models are trained once on the training split.

For all models, the classification threshold is selected on the validation split by maximizing the F_1 score over a quantile grid of candidate thresholds. The selected threshold is then fixed and applied to the test split.

We report AUC, F_1 , precision, and recall. The primary comparison metric is the mean test F_1 score averaged across cross-validation folds.

6.5. Large-Scale Benchmark Across 53 Datasets

To assess robustness across heterogeneous tasks, we adopt a ranking-based evaluation protocol. For each dataset, models are ranked according to their mean test F_1 score (rank 1 = best), and ranks are averaged across the 53 datasets. In addition to the mean rank, we report the number and rate of wins (rank = 1) and top-2 finishes (rank ≤ 2).

Linear benchmark. Table 1 summarizes the results for purely linear models. SGD achieves the best aggregate performance, attaining the lowest mean rank and the highest win frequency. It appears among the top two models on the vast majority of datasets, confirming its strong and stable behavior in standard linear settings.

Table 1

Ranking comparison for linear models across 53 OpenML datasets. Lower mean rank indicates better performance.

Model	Mean Rank ↓	Wins	Top-2	Win Rate	Top-2 Rate
SGD	1.660	26	45	0.491	0.849
Stripe-L (Linear)	2.113	16	31	0.302	0.585
Stripe-C (Linear)	2.151	14	31	0.264	0.585

Table 2

Ranking comparison across 53 OpenML datasets for the **kernel-based benchmark**. Lower mean rank indicates better performance.

Model	Mean Rank ↓	Wins	Top-2	Win Rate	Top-2 Rate
SVM (kernel)	1.962	32	35	0.604	0.660
Stripe-C (Kernel)	2.585	7	27	0.132	0.509
Stripe-L (Kernel)	2.585	7	25	0.132	0.472
RFF-SGD	2.736	10	22	0.189	0.415

The Stripe linear variants remain competitive across the benchmark. Both Stripe-L (Linear) and Stripe-C (Linear) achieve a substantial number of first-place finishes and consistent top-2 placements. Although their average ranks are moderately higher than that of SGD, their performance remains stable across a wide range of datasets.

These results indicate that, even in the absence of nonlinear feature mappings, Stripe provides a competitive alternative to conventional gradient-based linear optimization, while preserving its violation-driven and update-sparse training dynamics.

Kernel benchmark. To ensure a controlled comparison among **kernel-based approaches**, we compute ranks considering SVM (kernel), Stripe-L (kernel), Stripe-C (kernel), and RFF-SGD.

As shown in Table 2, kernel SVM achieves the strongest aggregate performance, obtaining the lowest mean rank and the highest number of wins.

The Stripe variants obtain intermediate mean ranks and achieve top-2 placements on approximately half of the datasets, indicating competitive performance on a substantial subset of tasks.

RFF-SGD exhibits a comparable overall profile, with a slightly higher mean rank but a non-negligible number of wins.

Overall, while kernel SVM provides the strongest average performance within this **kernel-based benchmark**, the Stripe systems demonstrate consistent performance across heterogeneous datasets under the same evaluation protocol.

6.6. Learning Dynamics and Update Sparsity

Stripe differs from gradient-based optimization in that parameter updates are performed only when constraint violations exceed a prescribed tolerance. This suggests that training should exhibit a violation-driven dynamic rather than continuous incremental adjustments.

To empirically examine this behavior, we analyze the number of corrective updates per epoch.

As shown in Figure 3, the majority of corrective updates occur during the initial training epochs. Following this early phase of intensive corrections, the update frequency decreases substantially and stabilizes.

This behavior is consistent with a violation-driven training mechanism: once most constraints are satisfied within the prescribed tolerance, further parameter adjustments become infrequent. In contrast to gradient-based optimization, which performs continuous updates throughout training, Stripe transitions to a sparse-update regime after an initial correction phase.

Such dynamics suggest that Stripe rapidly approaches a stable feasibility region, after which learning proceeds through occasional corrections rather than persistent incremental adjustments. This update sparsity may contribute to computational efficiency and reduced sensitivity to small stochastic fluctuations.

6.7. Ablation Study: Effect of the Parameter β (C-system)

The parameter β controls the magnitude of the corrective update within the buffer region $\epsilon \leq |\eta| < \epsilon + \delta$, thereby regulating the correction intensity near the boundary of admissibility.

To analyze its effect, we examine: (i) the evolution of predictive performance across training epochs, (ii) the total number of corrective updates, and (iii) the relationship between computational effort and final classification quality.

As shown in Figure 4, Stripe reaches a stable performance level within the first few epochs. Differences across β values are modest and primarily visible during the early training phase. In contrast, RFF-SGD exhibits a more gradual convergence pattern.

Figure 5 indicates that varying β leads to moderate changes in the total number of updates, without signs of instability or abrupt growth in computational cost. No clear monotonic relationship is observed between the number of updates and the final classification performance: configurations with substantially different update counts achieve comparable F_1 scores.

Overall, the results indicate that while β modulates the intensity of corrective steps and mildly affects the total number of updates, predictive performance remains comparatively stable across the examined range.

This suggests that the algorithm is not overly sensitive to the precise choice of β , and that variations in correction magnitude primarily influence training dynamics rather than the final decision boundary.

6.8. Ablation Study: Effect of the Parameter δ

The parameter δ determines the width of the buffer region between the strictly feasible zone and the region of full correction. It therefore controls the smoothness of the transition between no update and a full corrective projection.

To assess its impact, we analyze the evolution of predictive performance across epochs and the relationship between the number of corrective updates and the final classification quality.

As shown in Figure 6, the performance dynamics remain qualitatively similar across different δ values. Stripe reaches a stable F_1 level within the first few epochs, and differences between configurations remain moderate. This indicates an absence of fragile sensitivity with respect to the buffer width.

Figure 7 shows that varying δ affects the total number of corrective updates, yet no clear systematic relationship emerges between computational effort and final performance. Even configurations with noticeably different update counts achieve comparable F_1 scores.

Overall, the results indicate that while δ affects the frequency of corrective updates, predictive performance remains largely stable across the examined range.

This suggests that moderate variations in the buffer width primarily influence training dynamics rather than the final classification quality, and that the method does not exhibit fragile sensitivity to the precise choice of transition threshold.

6.9. Update dynamics

This experiment focuses on the training dynamics of the Stripe L -system rather than on final predictive performance. We analyze the epoch-wise correction behavior of the method on 45 selected binary tabular benchmarks. The purpose is to examine whether the correction effort is distributed approximately uniformly across epochs or is mainly concentrated in the early stages of training.

Figure 8 summarizes the pooled correction dynamics over dataset-repeat runs. For the L -system, the magnitude of coordinate corrections decreases rapidly during training. After normalization by the first-epoch value, the median correction magnitude drops from 1.000 at epoch 1 to 0.091 at epoch 5, 0.045 at epoch 10, 0.018 at epoch 20, and 0.009 at epoch 40. Thus, by the end of training, the typical correction magnitude is below 1% of its initial value.

The same effect is visible in the cumulative correction effort. The median cumulative effort already reaches 38.8% after the first epoch, 66.6% by epoch 5, 78.6% by epoch 10, and 88.8% by epoch 20. In contrast, a uniform-per-epoch reference would allocate only 2.5%, 12.5%, 25%, and 50% of the total effort to the same epochs. Therefore, the L -system is strongly front-loaded: most of the correction effort is spent in the early epochs, while later epochs mostly perform small refinements.

Thus, the main observation is not a uniform improvement in final accuracy, but a specific temporal structure of the L -system updates. The L -system concentrates most of its correction effort early in training and then continues with substantially smaller corrections.

Table 3

Final quality and run-to-run stability on the subset of 36 tabular benchmarks with all linear baselines available.

Model	Datasets	Quality		Run-to-run stability		Gap
		Test F_1	Test AUC	Run std. F_1	Run std. AUC	Train-test F_1
SGD	36	0.592	0.695	0.022	0.034	0.045
SVM-Linear	36	0.659	0.787	—	—	0.045
Stripe-L	36	0.579	0.653	0.006	0.008	0.063
Stripe-C	36	0.584	0.684	0.050	0.084	0.037

Boldface in the quality columns indicates the best overall value. In the run-to-run stability columns, boldface is used among iterative methods only. SVM-Linear is a deterministic batch baseline in this protocol; therefore, repeat variability is not reported for this model and is not directly comparable to the stochastic run-to-run variability of iterative training procedures.

6.10. Stability diagnostics

We next consider two complementary notions of stability on the complete subset of 36 selected tabular benchmarks where all linear baselines are available. The first one is *epoch-wise stability*: it measures the volatility of the validation-quality trajectory during training. The second one is *run-to-run stability*: it measures the variability of the final test quality across repeated runs for the same dataset and model.

For epoch-wise stability, we analyze only the iterative linear methods: SGD, Stripe-L, and Stripe-C. The batch SVM baseline is not included in this diagnostic because it does not have an epoch-wise training trajectory. We use validation F_1 trajectories for this diagnostic and reserve the test set for final held-out evaluation. For each dataset-repeat run, we compute three quantities from the validation F_1 trajectory: the standard deviation over the last ten epochs, the mean absolute epoch-to-epoch change, and the first epoch after which all subsequent validation F_1 values remain within 0.005 of the final validation value.

Figure 9 summarizes the epoch-wise validation-quality stability on this complete subset of 36 benchmarks. Stripe-L has the most stable validation trajectory among the iterative linear methods under these diagnostics. Its mean standard deviation over the last ten epochs is 0.002, compared with 0.022 for SGD and 0.055 for Stripe-C. The mean absolute epoch-to-epoch change is also smallest for Stripe-L: 0.003, compared with 0.024 for SGD and 0.058 for Stripe-C. Finally, Stripe-L reaches the stability tolerance earlier on average, at epoch 10.7, whereas SGD and Stripe-C reach it at epochs 23.1 and 36.6, respectively.

The distribution over dataset-repeat runs supports the same qualitative pattern, especially for last-epoch variability and epoch-to-epoch changes. Stripe-L has substantially lower validation-trajectory volatility, while Stripe-C exhibits larger fluctuations. This is consistent with the update-dynamics analysis: Stripe-L concentrates most of its correction effort in the early epochs and then continues with small refinements. Thus, the epoch-wise stability advantage is specific to the L -system variant and should not be interpreted as a uniform property of both Stripe formulations.

Table 3 reports final test quality and run-to-run stability on the same subset of 36 benchmarks. In this table, we also include the batch SVM baseline, because run-to-run stability is a final-quality diagnostic and does not require an epoch-wise trajectory. Run-to-run stability is computed as the standard deviation of the final test F_1 across repeats for each dataset and model, followed by averaging over datasets. Since SVM-Linear is a deterministic batch baseline under a fixed train/validation/test split, its zero repeat variability should be interpreted as a batch reference rather than compared directly with the stochastic repeat-to-repeat variability of the iterative training procedures.

The final-quality comparison shows that SVM-Linear obtains the highest mean test F_1 on this subset (0.659). Among the iterative methods, the mean test F_1 scores are comparable: 0.592 for SGD, 0.579 for Stripe-L, and 0.584 for Stripe-C. However, their run-to-run variability differs substantially. Stripe-L has the lowest mean run-to-run standard deviation of test F_1 among the iterative methods (0.006), compared with 0.022 for SGD and 0.050 for Stripe-C.

Overall, the stability diagnostics show that Stripe-L is the most stable among the iterative linear methods in this comparison with respect to validation-trajectory volatility and final run-to-run variability. At the same time, this stability does not imply the highest final predictive accuracy: SVM-Linear remains the strongest method in mean test F_1 on this subset. Stripe-C, in contrast, should not be interpreted as a uniformly more stable variant. Its validation trajectory is more volatile in this experiment, which is consistent with the fact that the C -system correction mechanism depends directly on pointwise constraint violations and on the tolerance parameter ϵ . This observation motivates a

separate structural diagnostic of ε -sensitivity, considered in Subsection 6.1.1, where we examine whether the variability of the C -system can be related to properties of the training set.

6.11. Structural diagnostic of ε -sensitivity

The tolerance parameter ε in the C -system controls the width of the admissible stripe around each training constraint. In practice, the influence of this parameter may vary substantially across datasets. In this section we investigate whether this variation can be partially explained by structural properties of the training set.

Let $y_i \in -1, +1$ and let the linear model be $f_\theta(x_i) = \langle x_i, \theta \rangle$. For sample i , define the residual $\eta_i(\theta) = \langle x_i, \theta \rangle - y_i$. The C -system constraint requires $|\eta_i(\theta)| < \varepsilon$.

When this constraint is violated, the algorithm performs a correction step. In the notation used here, this step can be written as $\theta^+ = \theta - \frac{\Psi_\varepsilon(\eta_i)}{|x_i|^2} x_i$, where $\Psi_\varepsilon(\eta_i)$ is the signed correction term determined by the C -system update rule. For a large violation, $\Psi_\varepsilon(\eta_i) = \eta_i$, and the update projects the active sample onto the central hyperplane of the stripe, since $\eta_i(\theta^+) = \eta_i(\theta) - \eta_i(\theta) = 0$.

The same correction also affects the residuals of other samples. For another sample j , we have $\eta_j(\theta^+) - \eta_j(\theta) = -\frac{\langle x_j, x_i \rangle}{|x_i|^2} \Psi_\varepsilon(\eta_i)$. This motivates the directed interaction coefficient $G_{ji} = \frac{\langle x_j, x_i \rangle}{|x_i|^2 + \rho}$, where $\rho > 0$ is a small stabilizing constant. Then $\Delta\eta_j \approx -G_{ji} \Psi_\varepsilon(\eta_i)$. Thus, $|G_{ji}|$ measures how strongly a correction performed for sample i propagates to sample j .

However, the magnitude of G_{ji} alone does not determine whether this interaction is beneficial or harmful. A strong interaction may be aligned with the class labels, in which case a correction reducing the residual of one sample can also improve other samples. Conversely, a strong interaction may be label-conflicting, in which case a correction reducing the residual of one sample may deteriorate the signed margin or residual of another sample.

To distinguish these two cases, we consider the signed interaction $A_{ji} = G_{ji} y_i y_j$. Positive values of A_{ji} correspond to interactions that are geometrically aligned with the label structure, whereas negative values indicate potentially label-conflicting interactions. Therefore, A_{ji} should be interpreted as a structural diagnostic of whether correction propagation is likely to be aligned or conflicting with the labels. Importantly, this condition is not equivalent to simply checking whether two labels are equal. For example, if $y_i = y_j$ and $G_{ji} > 0$, the interaction is aligned. If $y_i \neq y_j$ and $G_{ji} < 0$, then $G_{ji} y_i y_j > 0$ as well, and the geometry is also aligned with class separation. A conflict occurs when the sign of the geometric interaction is inconsistent with the product of the labels.

Since weak interactions are unlikely to dominate the update dynamics, we focus on the strongest off-diagonal interactions. Let $\mathcal{T} * 0.1$ denote the set of pairs (j, i) for which $|G * ji|$ lies above the 90th percentile among all off-diagonal interactions. Due to possible ties, this corresponds approximately to the largest 10

The corresponding top structural conflict statistic is $S_{\text{top}}(G, y) = |\bar{G}_{ji}| * (j, i) \in \mathcal{T} * 0.1 \cdot C_{\text{top}}(G, y)$. Equivalently, this statistic can be written as $S_{\text{top}}(G, y) = \frac{1}{|\mathcal{T} * 0.1|} \sum_{(j, i) \in \mathcal{T} * 0.1} [-G * jiy_j]_+$. This statistic combines two factors: the strength of the dominant interactions and their disagreement with the labels.

Our hypothesis is that the C -system becomes more sensitive to ε not merely when samples are strongly coupled, but when the dominant interactions contain a substantial label-conflicting component. In such cases, changing ε changes the set and intensity of active corrections, and these corrections may interfere with each other through strong conflicting interactions.

Experimental protocol. We evaluated this hypothesis on 67 binary OpenML datasets. All structural quantities were computed only on the training set before fitting the model. For each dataset, we computed the interaction matrix G , the signed interactions $A_{ji} = G_{ji} y_i y_j$, the top-conflict statistic $S_{\text{top}}(G, y)$, and several control structural measures: mean absolute interaction strength, conflict score over all off-diagonal pairs, structural conflict over all pairs, and the effective Gram condition number.

After computing the structural characteristics, we trained the linear C -system over the fixed grid $\varepsilon \in 0.01, 0.02, 0.03, 0.05, 0.0$. The remaining C -system parameters, including δ -ratio, β , regularization, and initialization mode, were tuned separately using only the training/validation protocol at a fixed reference value of ε , while the test set was kept for final evaluation. Thus, the experiment measured the effect of explicitly varying ε , rather than the effect of simultaneously changing all hyperparameters.

For each value of ε , the model was trained for a fixed number of epochs. The classification threshold was selected on the validation set and then applied to the test set. We measured sensitivity to ε as the range of the F_1 -score across the grid: $\text{Sens} * \text{val} = \max_{\varepsilon} F_1^{\text{val}}(\varepsilon) - \min_{\varepsilon} F_1^{\text{val}}(\varepsilon)$ and $\text{Sens} * \text{test} = \max_{\varepsilon} F_1^{\text{test}}(\varepsilon) - \min_{\varepsilon} F_1^{\text{test}}(\varepsilon)$. Small

Table 4Correlation between structural characteristics and validation sensitivity to ϵ across 67 dataset-level observations.

Structural measure	n	Spearman ρ	p -value
S_{top} without bias	67	0.420	0.000403
S_{top} with bias	67	0.402	0.000751
Structural conflict, all pairs	67	0.217	0.0775
Mean interaction strength	67	0.189	0.125
Effective Gram condition number	67	0.097	0.436
Conflict score, all pairs	67	-0.029	0.818

Table 5Correlation between structural characteristics and test sensitivity to ϵ across 67 dataset-level observations.

Structural measure	n	Spearman ρ	p -value
S_{top} without bias	67	0.374	0.001831
S_{top} with bias	67	0.369	0.002150
Structural conflict, all pairs	67	0.138	0.266
Mean interaction strength	67	0.135	0.276
Effective Gram condition number	67	-0.020	0.872
Conflict score, all pairs	67	-0.045	0.715

Table 6Sensitivity to ϵ grouped by top structural conflict S_{top} .

S_{top} range	n	Val mean	Val median	Test mean	Test median
$S_{\text{top}} < 0.30$	28	0.105	0.094	0.136	0.109
$0.30 \leq S_{\text{top}} < 1.00$	25	0.142	0.091	0.182	0.151
$S_{\text{top}} \geq 1.00$	14	0.217	0.203	0.233	0.231

values of these quantities indicate that the final quality is relatively insensitive to the choice of ϵ , whereas large values indicate that the tolerance parameter substantially affects the result.

Each dataset was evaluated with several training repeats. However, the final correlation analysis was performed at the dataset level. This is important because G , S_{top} , and the effective Gram condition number are identical across repeats of the same dataset. Therefore, repeat-level results were first aggregated to dataset-level observations before computing correlations.

Results. The experiment was successfully completed on all 67 binary OpenML datasets. We examined the relationship between structural characteristics of the training set and sensitivity to ϵ . The effective Gram condition number, used as a global measure of interaction conditioning, showed almost no rank correlation with sensitivity: $\rho_{\text{Spearman}} \approx 0.097$ for validation sensitivity and $\rho_{\text{Spearman}} \approx -0.020$ for test sensitivity.

In contrast, the top structural conflict statistic $S_{\text{top}}(G, y)$ showed a substantially stronger rank association with sensitivity. Its Spearman correlation with validation sensitivity was $\rho_{\text{Spearman}} \approx 0.420$, $p \approx 0.000403$, and its correlation with test sensitivity was $\rho_{\text{Spearman}} \approx 0.374$, $p \approx 0.001831$. The same qualitative pattern was observed for the version of the statistic computed with the bias term included: the corresponding Spearman correlations were approximately 0.402 for validation sensitivity and 0.369 for test sensitivity. By contrast, structural conflict computed over all pairs, mean interaction strength, and conflict score over all pairs showed weaker associations with sensitivity.

As an additional descriptive check, we grouped datasets by the value of S_{top} : $S_{\text{top}} < 0.30$, $0.30 \leq S_{\text{top}} < 1.00$, and $S_{\text{top}} \geq 1.00$. The results are shown in Table 6. The mean validation sensitivity increased from 0.105 in the low- S_{top} group to 0.217 in the high- S_{top} group. The mean test sensitivity increased from 0.136 to 0.233. This descriptive grouping is consistent with the correlation analysis: datasets with stronger label-conflicting dominant interactions tend to exhibit greater sensitivity to ϵ .

Interpretation. These results do not imply that S_{top} determines the optimal value of ϵ . The experiment is correlational and should be interpreted as an empirical diagnostic rather than as a causal theorem. Nevertheless, the results support

the following interpretation: the tolerance parameter ε becomes important not merely when samples are strongly coupled geometrically, but when the strongest couplings contain a substantial label-conflicting component.

Global interaction strength or conditioning describes the potential magnitude of correction propagation, but it does not distinguish whether this propagation is aligned or conflicting with respect to the labels. The signed top-conflict statistic $S_{\text{top}}(G, y)$ captures this distinction and is therefore more informative as a pre-training diagnostic of potential ε -sensitivity.

Practically, low values of S_{top} suggest that the model is less likely to be fragile with respect to the tolerance choice, so a coarser ε -grid may be sufficient. High values of S_{top} warn that the tolerance parameter should be tuned more carefully, because the final quality may change substantially across the grid.

Thus, the experiment supports the refined view that the influence of ε is not purely arbitrary, but is related to the structure of the training set, and in particular to the presence of strong label-conflicting interactions.

6.12. Discussion

The experimental results support several key observations.

First, Stripe demonstrates competitive classification performance across heterogeneous tabular datasets. Within the considered set of linear and **kernel-based baselines**, it consistently attains intermediate aggregate ranks and, on a subset of datasets, achieves top-ranked positions. While kernel SVM remains the strongest method on average in the **kernel-based benchmark**, Stripe maintains consistent performance across datasets under the same evaluation protocol.

Second, the analysis of training dynamics shows that the majority of corrective updates occur during the early epochs, after which the process stabilizes. As training progresses, updates become increasingly sparse. This behavior is consistent with interpreting Stripe as a violation-driven method that performs corrections primarily to restore feasibility, rather than continuously optimizing a smooth global objective.

Third, the ablation studies indicate that predictive performance remains stable under moderate variations of the structural parameters β and δ . Although these parameters influence the frequency and intensity of corrective updates, no clear systematic relationship emerges between the total number of updates and generalization performance. This indicates that the method does not rely on narrowly tuned hyperparameter regimes.

Taken together, these findings characterize Stripe as a constraint-oriented alternative to gradient-based training, based on projection-like corrections of constraint violations. The empirical behavior is consistent with a finite-update regime once approximate feasibility is reached.

Stripe and RFF-SGD rely on fundamentally different training principles. SGD performs incremental updates at every step and depends on learning-rate scheduling, whereas Stripe applies corrections only when violations occur and does not employ a learning rate in the conventional gradient-descent sense. Empirically, Stripe exhibits earlier stabilization of validation performance and a structured update pattern, while achieving comparable F_1 scores relative to RFF-SGD across datasets.

We additionally examined performance in small-sample regimes ($n < 1000$). In this setting, Stripe increases its win rate against SVM from 37.7% to 46.7%, narrowing the performance gap. While the average rank remains comparable, this suggests that projection-based dynamics remain viable when the number of observations is limited.

In addition to competitive mean performance, Stripe often exhibits lower variability of validation F_1 across epochs and folds in early training phases, reflecting more stable convergence behavior.

6.13. Limitations

Our study has several limitations.

Feasibility assumptions and noisy regimes. Classical finite-update guarantees for Stripe-type methods rely on strict feasibility of the underlying hyperslab system. In our experiments, training is performed in an epoch-wise regime on a fixed finite dataset; consequently, both the L - and C -formulations induce finite systems of constraints. However, finiteness of the constraint set does not imply strict feasibility. In practical learning problems with label noise, outliers, or model mismatch, there may exist no parameter vector satisfying all constraints with a positive margin. In such cases, the classical finite-termination interpretation of Yakubovich does not directly apply, and Stripe should instead be viewed as a violation-driven row-action method applied to a finite (possibly inconsistent) system. Although the ε -insensitive and multizone variants mitigate oscillations and improve robustness in such regimes, a systematic theory and principled parameter selection for explicitly infeasible systems remain open problems.

Hyperparameter selection and comparability to baselines. Stripe replaces learning-rate tuning with structural parameters such as ϵ , δ , and β (and regularization or decay when enabled). Although our ablation studies suggest moderate sensitivity, these parameters still require calibration and can affect both predictive accuracy and the number of corrective updates, thereby influencing runtime. In addition, our evaluation relies on a limited hyperparameter-optimization budget, and relative rankings may shift under more extensive tuning.

Kernel scalability. The kernelized variants require storing and updating Gram-type structures whose memory and computational costs grow with the dictionary size. In the simplest batch setting (dictionary equal to all training points), this results in quadratic memory growth and may limit applicability to large-scale datasets. While caching and epoch-wise updates reduce computational overhead, scalable dictionary management strategies, budget maintenance schemes, or low-rank approximations are required for larger problems.

Scope of models and tasks. We focus on binary classification with linear models and RBF-kernel expansions. Extensions to regression and multiclass classification are conceptually straightforward within the same projection-based framework and can be implemented using appropriate constraint formulations (e.g., one-vs-rest schemes or task-specific tolerance constraints). However, these extensions are not explored empirically in the present study and are left for future work.

7. Conclusion and Future Work

In this work, we revisited Yakubovich’s Stripe algorithms from the perspective of modern supervised learning and formulated them as projection-based methods for solving systems of hyperslab constraints. We showed that both relaxed optimality conditions (L -systems) and pointwise tolerance constraints (C -systems) naturally lead to a unified projection framework in which parameter updates are triggered exclusively by constraint violations.

This formulation connects classical Stripe algorithms with contemporary machine learning paradigms, clarifies their relation to row-action methods such as the Kaczmarz algorithm, and adapts classical finite-update guarantees to the supervised learning setting under strict feasibility assumptions.

We further introduced a kernel extension that transfers hyperslab constraints and projection updates to the space of kernel expansion coefficients, enabling nonlinear decision functions while preserving the geometric structure of the algorithm.

Experimental evaluation on a diverse collection of OpenML tabular datasets demonstrates that Stripe achieves competitive predictive performance relative to strong linear baselines. The results reveal a characteristic violation-driven training dynamic, rapid stabilization of updates, and moderate sensitivity to structural parameters such as β and δ .

At the same time, Stripe is a feasibility-oriented method rather than a probabilistic model, and its performance depends on appropriate tolerance selection. While tree-based ensembles often attain higher aggregate rankings, Stripe provides a geometrically interpretable and update-sparse alternative to gradient-based optimization.

A systematic numerical comparison between Stripe algorithms, YAVA/perceptron-type procedures, and classical optimization solvers such as least-squares and simplex-type methods remains an interesting direction for future work, since these approaches correspond to different but closely related problem formulations.

Future research directions include theoretical analysis in explicitly noisy and infeasible regimes, adaptive selection of tolerance parameters, and the exploration of projection-based training in more complex model classes.

A particularly promising direction is the use of Stripe-type row-action updates as inner solvers in Gauss–Newton and Natural Gradient methods. Recent work Goldshlager, Hu and Lin (2025) has shown that randomized block Kaczmarz and related solvers can efficiently handle the mini-batch linear least-squares subproblems arising in such settings. In this context, each row corresponds to a hyperplane constraint of the form $j_i^\top \Delta\theta = r_i$, and classical row-action methods enforce exact satisfaction of these constraints. Replacing hyperplane projections with stripe constraints of the form $|j_i^\top \Delta\theta - r_i| \leq \epsilon \|j_i\|$ would yield a tolerance-aware row-action least-squares solver with a geometrically interpretable dead-zone parameter ϵ . Such an approach may help suppress small inconsistencies caused by stochastic noise and linearization error in neural network training, while preserving the computational advantages of row-wise updates.

Acknowledgements

Do not include acknowledgements in the initial version of the paper submitted for blind review.

Impact Statement

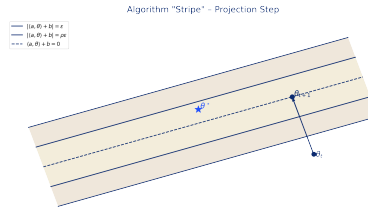


Figure 1: Basic Stripe algorithm: projection of θ_k onto the central hyperplane of the hyperslab.

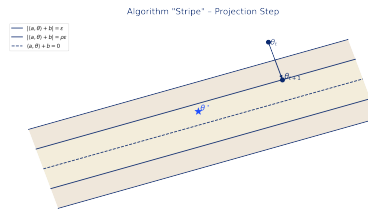


Figure 2: Improved Stripe algorithm ($\mu_k = 1$): projection onto the nearest boundary of the hyperslab.

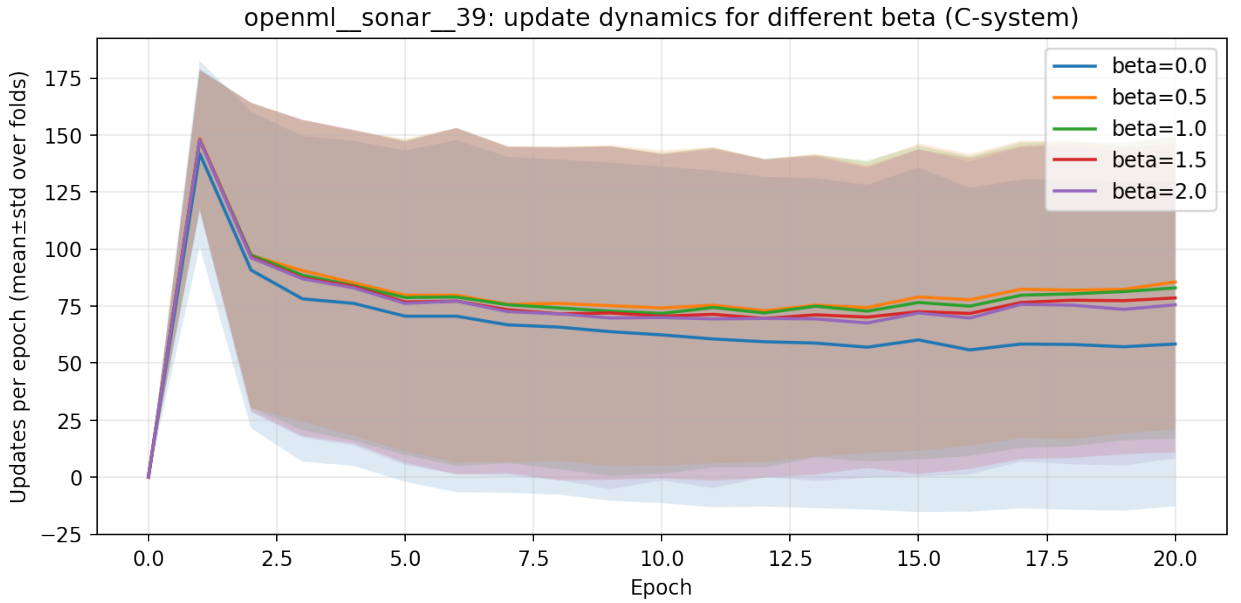


Figure 3: Corrective updates per epoch (mean $\pm$ std) for different β values (C-system).

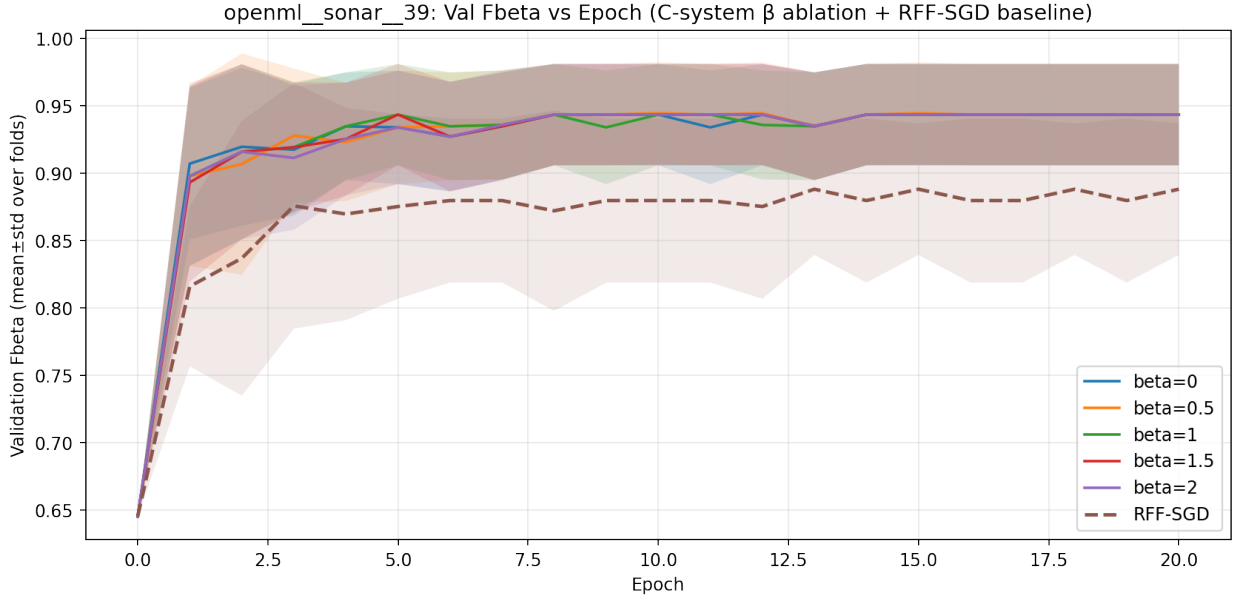


Figure 4: Validation F_1 as a function of training epochs for different values of β (C-system), with RFF-SGD shown for reference.

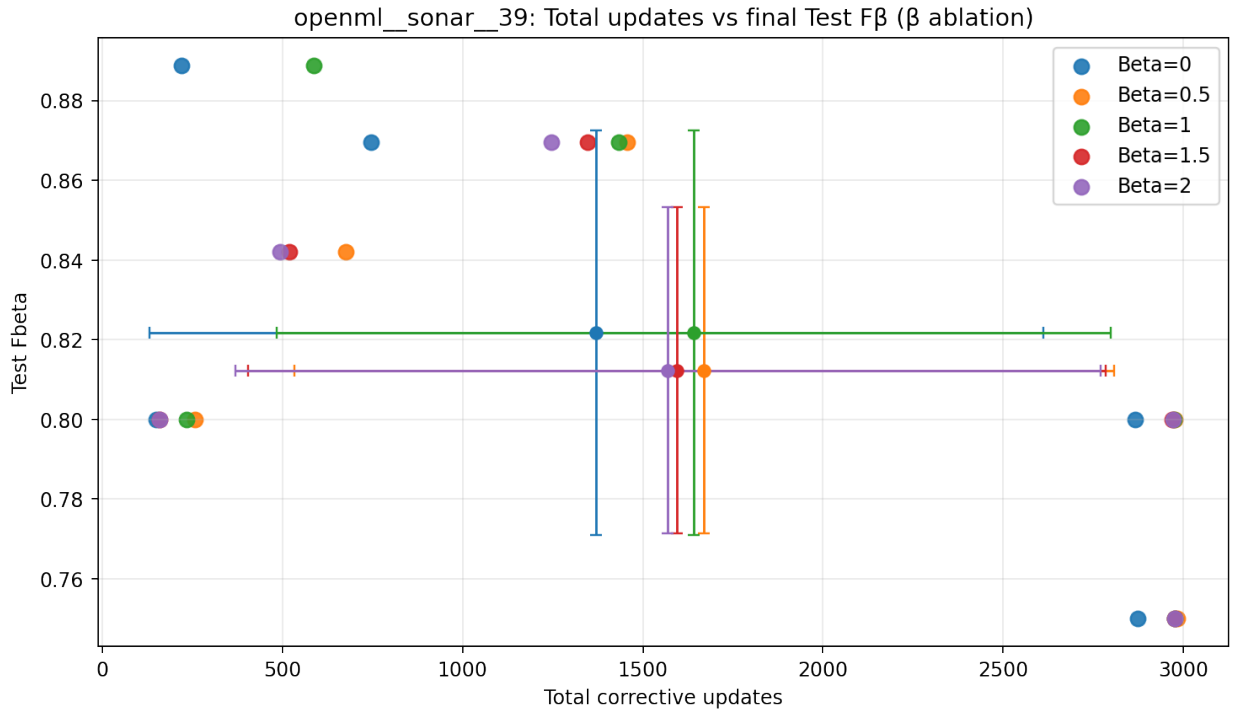


Figure 5: Total number of corrective updates and final test F_1 for different values of β (mean $\pm$ std).

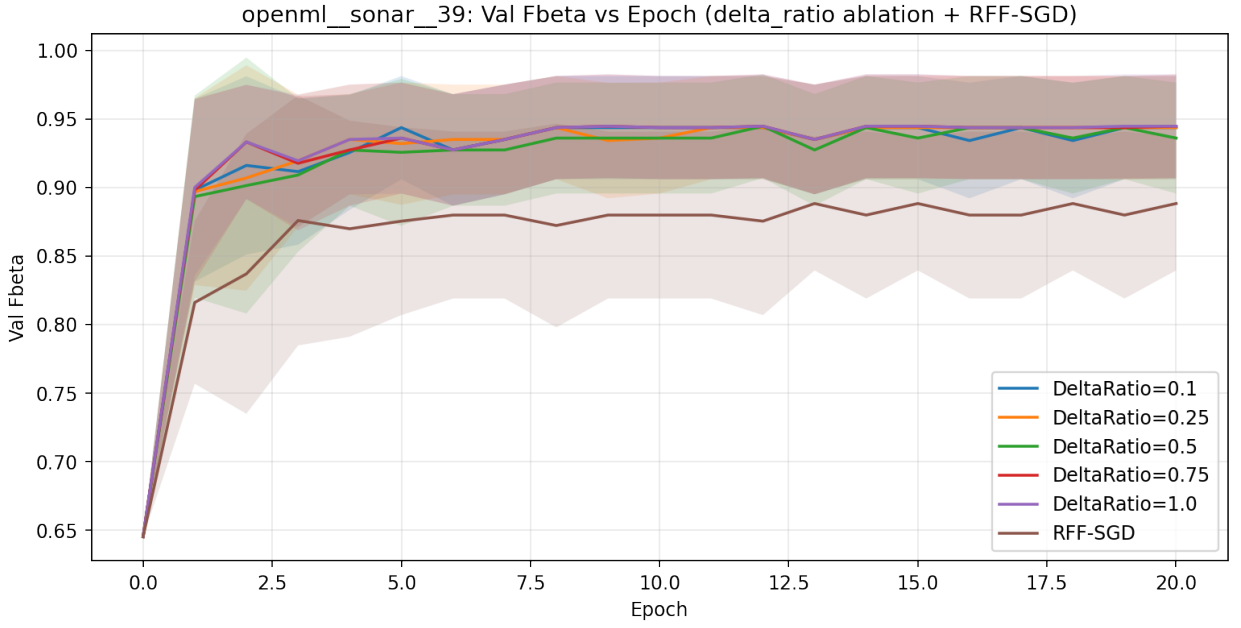


Figure 6: Validation F_1 as a function of training epochs for different values of δ (C-system).

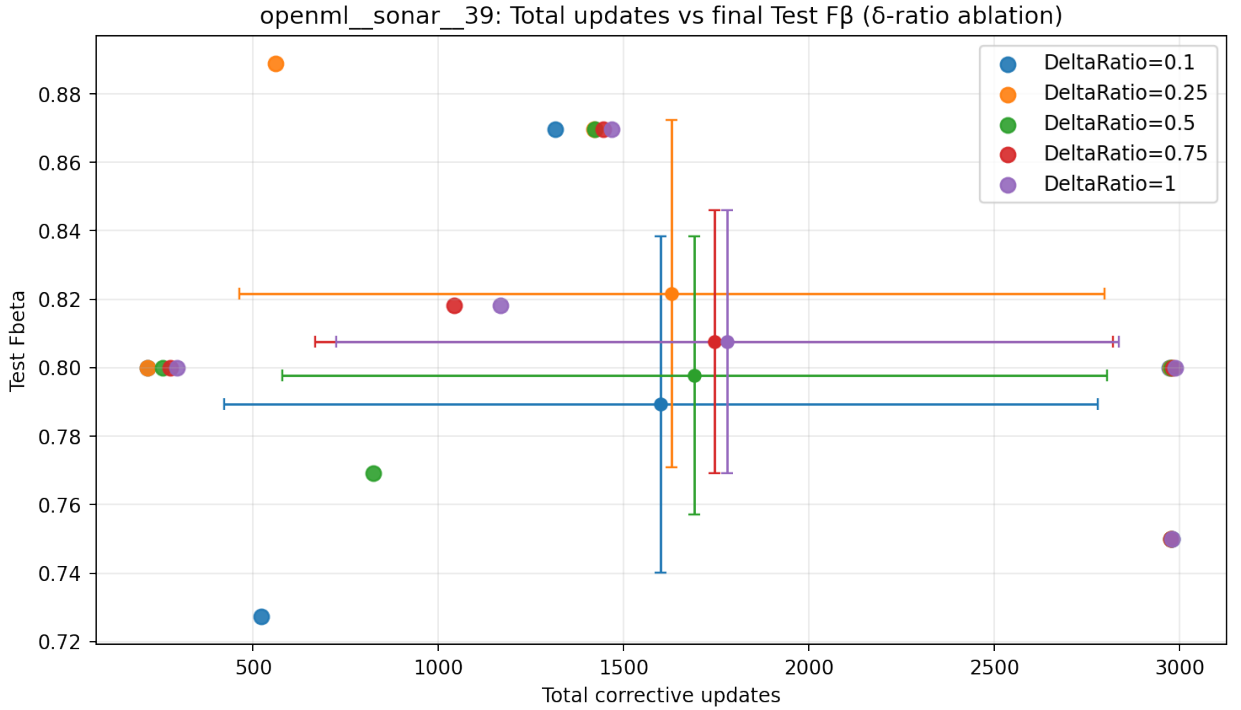


Figure 7: Total number of corrective updates and final test F_1 for different values of δ (mean $\pm$ std).

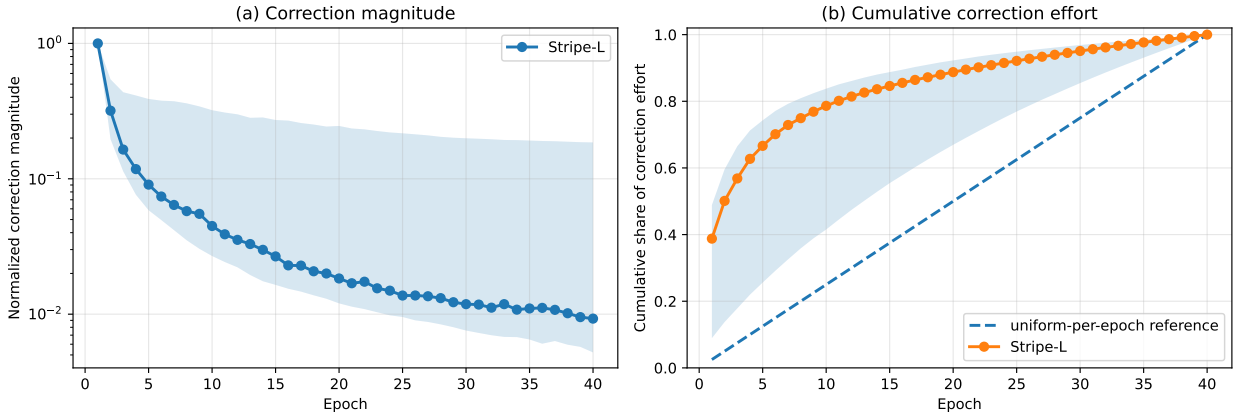


Figure 8: Correction-effort dynamics of the Stripe L -system on 45 selected tabular benchmarks. Panel (a) shows the L -system correction magnitude, normalized by its first-epoch value, on a logarithmic scale. Panel (b) shows the cumulative share of the total correction effort. Correction effort is measured by the sum of correction magnitudes within an epoch. The dashed line denotes the uniform-per-epoch reference. Curves show medians over dataset-repeat runs; shaded regions show interquartile ranges. **The figure should be interpreted as a diagnostic comparison of update behavior, not as evidence that Stripe uniformly improves final predictive accuracy over all baselines.**

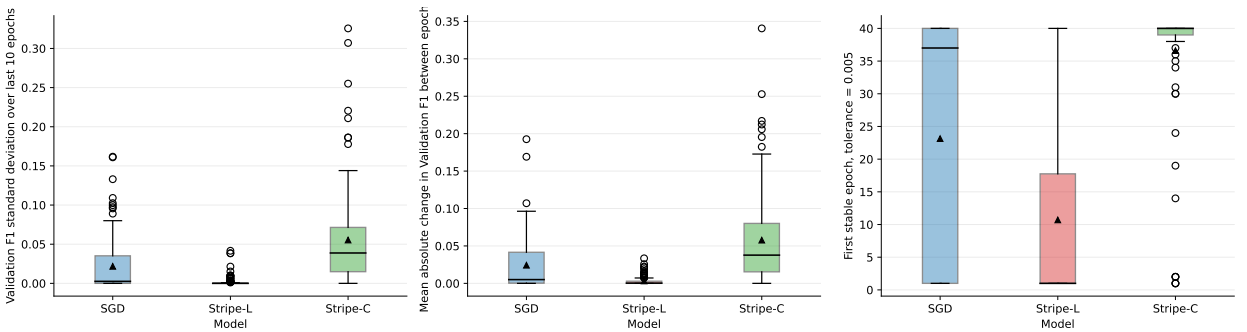


Figure 9: Epoch-wise validation-quality stability on the complete subset of 36 selected tabular benchmarks. The diagnostics are computed over dataset-repeat runs for the iterative methods only. The three panels show: the standard deviation of validation F_1 over the last ten epochs, the mean absolute epoch-to-epoch change in validation F_1 , and the first epoch after which the trajectory remains within 0.005 of its final validation value.

A. Appendix

B. Computational Aspects and Implementation

This section summarizes the implementation details of the proposed Stripe-based methods. Across all variants, we distinguish between (i) the construction and update of the constraint system from data and (ii) the projection-based correction steps that enforce feasibility up to a prescribed tolerance.

B.1. Linear L -system (online and cumulative settings)

The linear L -system is implemented using cumulative statistics. Given samples (x_i, s_i) (with $s_i \in \{-1, +1\}$ in classification), the algorithm maintains running estimates of the empirical second-order matrix and the correlation vector, $A^{(m)} \approx \frac{1}{m} \sum_{i=1}^m x_i x_i^\top$, $b^{(m)} \approx \frac{1}{m} \sum_{i=1}^m s_i x_i$. In the implementation, these statistics are updated online using standard averaging recursions. After each update, a small number of Stripe projection steps is performed over parameter coordinates (i.e., by iterating over rows $A_h^{(m)}$).

Operating modes. The implementation supports three practical operating modes: *single* (statistics constructed from the current sample only), *cumulative* (statistics averaged over all observed samples), and *epoch-wise* passes over a fixed dataset with shuffling. These modes differ only in how the running statistics are updated; the projection step itself remains unchanged.

Bias term. When enabled, a constant feature is appended to each input vector, allowing the bias parameter to be learned jointly with the weights.

B.2. Linear C -system (multizone correction)

The linear C -system enforces pointwise constraints on the prediction error. For a signed label $s_i \in \{-1, +1\}$ and prediction $p_i = \langle w, x_i \rangle$, the residual is defined as $\eta_i = p_i - s_i$. A piecewise multizone correction rule is applied, controlled by the tolerance $\varepsilon > 0$, buffer width $\delta > 0$, and relaxation parameter β . The update is activated only when $|\eta_i| \geq \varepsilon$ and follows a three-zone scheme: projection onto the centerline for large violations, reflection near the boundary, and a relaxed correction within the buffer zone.

Regularization. The linear C -system may optionally be stabilized using multiplicative weight decay. In the current implementation, weight decay is applied after processing each individual constraint, which preserves the explicit and modular structure of the projection-based update.

B.3. Kernelized L -system

The kernelized L -system is implemented in the space of expansion coefficients $\alpha \in \mathbb{R}^M$. As new samples arrive, an implicit analogue of the Gram statistics is constructed via kernel Gram matrices. Between data acquisition steps, optimization proceeds in an epoch-wise manner: one epoch corresponds to a full pass over all current constraints with random reshuffling. To reduce computational overhead, system matrices and row norms are cached and recomputed only when new samples are added.

B.4. Kernelized C -system

The kernelized C -system is likewise implemented in the coefficient space α . Each new sample extends the kernel Gram matrix and introduces a new inequality constraint. Optimization proceeds epoch-wise by iterating over constraints in random order and applying the same multizone correction rule as in the linear case, with optional multiplicative decay applied prior to constraint evaluation. This preserves the geometric interpretation of the updates as projection-like corrections within a tolerance band.

All reported experiments employ feature scaling prior to training (and, when enabled, an appended bias feature), since the step size of projection-based methods is sensitive to feature norms.

References

Bergstra, J., Bardenet, R., Bengio, Y., Kégl, B., 2011. Algorithms for hyper-parameter optimization, in: Shawe-Taylor, J., Zemel, R., Bartlett, P., Pereira, F., Weinberger, K. (Eds.), *Advances in Neural Information Processing Systems*, Curran Associates, Inc. URL: https://proceedings.neurips.cc/paper_files/paper/2011/file/86e8f7ab32cfd12577bc2619bc635690-Paper.pdf.

- Bischl, B., Casalicchio, G., Feurer, M., Gijsbers, P., Hutter, F., Lang, M., Mantovani, R.G., van Rijn, J.N., Vanschoren, J., 2021. Openml benchmarking suites. URL: <https://arxiv.org/abs/1708.03731>, arXiv:1708.03731.
- Bishop, C.M., 2006. Pattern Recognition and Machine Learning. Springer.
- Detassis, F., Lombardi, M., Milano, M., 2021. Teaching the old dog new tricks: Supervised learning with constraints. URL: <https://arxiv.org/abs/2002.10766>, arXiv:2002.10766.
- Fomin, V.N., 1976. Mathematical Theory of Learning Pattern Recognition Systems. Leningrad State University Press, Leningrad. In Russian.
- Fomin, V.N., Fradkov, A.L., Yakubovich, V.A., 1981. Adaptive Control of Dynamical Systems. Nauka, Moscow. In Russian.
- Fradkov, A., Shepeljavyi, A., 2022. The history of cybernetics and artificial intelligence: a view from saint petersburg. Cybernetics and Physics , 253–263doi:10.35470/2226-4116-2022-11-3-253-263.
- Gijsbers, P., LeDell, E., Thomas, J., Poirier, S., Bischl, B., Vanschoren, J., 2019. An open source automl benchmark. URL: <https://arxiv.org/abs/1907.00909>, arXiv:1907.00909.
- Goldshlager, G., Hu, J., Lin, L., 2025. Worth their weight: Randomized and regularized block kaczmarz algorithms without preprocessing. arXiv preprint arXiv:2502.00882 .
- Gusev, S.V., Bondarko, V.A., 2020. Notes on yakubovich's method of recursive objective inequalities and its application in adaptive control and robotics. IFAC-PapersOnLine 53, 1379–1384. URL: <https://www.sciencedirect.com/science/article/pii/S240589632032499X>, doi:<https://doi.org/10.1016/j.ifacol.2020.12.1885>. 21st IFAC World Congress.
- Hastie, T., Tibshirani, R., Friedman, J., 2009. The Elements of Statistical Learning: Data Mining, Inference and Prediction. 2nd ed., Springer.
- Hofmann, T., Schölkopf, B., Smola, A.J., 2008. Kernel methods in machine learning. The Annals of Statistics 36. URL: <http://dx.doi.org/10.1214/009053607000000677>, doi:10.1214/009053607000000677.
- Kaczmarz, S., 1937. Angenäherte auflösung von systemen linearer gleichungen. Bulletin International de l'Académie Polonaise des Sciences et des Lettres. Classe des Sciences Mathématiques et Naturelles. Série A, Sciences Mathématiques 35, 355–357.
- Lipkovich, M., 2022. Yakubovich's method of recursive objective inequalities in machine learning. IFAC-PapersOnLine 55, 138–143. URL: <https://www.sciencedirect.com/science/article/pii/S2405896322007029>, doi:<https://doi.org/10.1016/j.ifacol.2022.07.301>. 14th IFAC Workshop on Adaptive and Learning Control Systems ALCOS 2022.
- Lipkovich, M.M., Sagatdinov, A.R., 2023. Application of the “stripe” algorithm for online decoding of the eeg patterns. Mekhatronika, Avtomatizatsiya, Upravlenie 24, 300–306. doi:10.17587/mau.24.300-306. in Russian.
- Peters, B., 2023. Cqnet: convex-geometric interpretation and constraining neural-network trajectories. URL: <https://arxiv.org/abs/2302.10895>, arXiv:2302.10895.
- Schölkopf, B., Smola, A.J., 2002. Learning with kernels : support vector machines, regularization, optimization, and beyond. Adaptive computation and machine learning, MIT Press. URL: <http://www.worldcat.org/oclc/48970254>.
- Yakubovich, V.A., 1965. Some general theoretical principles of construction of learning systems for pattern recognition. Computing Technique and Programming Questions (Vychislitelnaya Tekhnika i Voprosy Programmirovaniya) 4, 3–71.
- Yakubovich, V.A., 1966. Recurrent finitely convergent algorithms for solving systems of inequalities. Soviet Mathematics 7, 300–304.